**1. Verificação de vetor em ordem crescente**

O primeiro exercício proposto foi a implementação de um procedimento chamado e_crescente, cujo objetivo era verificar se os elementos de um vetor inteiro estavam estritamente ordenados em ordem crescente. Isso significa que, para cada posição do vetor, o elemento atual deveria ser menor que o próximo, sem permitir repetições ou igualdades. O procedimento deveria receber dois argumentos: o ponteiro para o início do vetor (em a0) e o tamanho total do vetor (em a1). O retorno deveria ser 1 (em a0) caso o vetor estivesse corretamente ordenado, ou 0 caso contrário.

O desafio inicial foi percorrer corretamente o vetor usando apenas instruções do conjunto básico, como add, lw, bge, blt, addi, jal e jalr, sem recorrer a pseudo-instruções como mv ou ecall. Um erro comum era ultrapassar os limites do vetor ou comparar além do penúltimo elemento, então foi preciso tomar cuidado para parar no elemento n-2 para comparar com o n-1. Também implementamos a lógica para considerar vetores com tamanho 0 ou 1 como sempre crescentes.

Os testes foram feitos diretamente no simulador Venus, com diferentes configurações de vetores: um vetor crescente (por exemplo, 1, 3, 5, 7, 9), um com valores repetidos (1, 3, 3, 7, 9) e um decrescente (9, 7, 5, 3, 1). Em todos os casos, a função respondeu corretamente com 1 ou 0, o que validou a lógica de comparação e percorrimento sequencial.

Esse exercício foi importante para entender melhor como trabalhar com vetores em Assembly, como gerenciar o acesso à memória usando ponteiros e offsets, e como controlar fluxos condicionais de forma segura.

**2. Atualização do valor do estoque**

O segundo problema proposto foi mais elaborado e envolvia a criação de três procedimentos distintos e encadeados: soma_estoque, ajustar_preco e aplicar_taxa. A ideia era simular o cálculo de valor real de um estoque, onde cada item tinha seu preço defasado multiplicado por um fator de aumento (3x), e em seguida uma taxa adicional de 50% era aplicada (multiplicação por 2).

A função soma_estoque percorre um vetor de preços utilizando um índice e, para cada elemento, chama ajustar_preco, passando o valor atual e o fator. A função ajustar_preco faz a multiplicação do valor original pelo fator e em seguida chama aplicar_taxa. A última, aplicar_taxa, também faz a multiplicação usando o valor da taxa. Cada uma dessas funções salva e restaura corretamente o registrador ra na pilha, garantindo que a pilha de chamadas não seja corrompida.

Um ponto crítico desse exercício foi garantir que as chamadas fossem feitas de fato. Em tentativas anteriores, era tentador fazer os cálculos diretamente em soma_estoque, mas isso violaria a regra da chamada aninhada. Depois de organizar o código de forma a separar claramente as responsabilidades de cada função, os testes passaram corretamente. A estrutura final está clara: main chama soma_estoque, que chama ajustar_preco, que chama aplicar_taxa.

Os testes foram automáticos e baseados em três chamadas diferentes.

**3. Considerações finais**

Esse trabalho ajudou a consolidar vários conceitos importantes em Assembly, como acesso à memória com ponteiros, uso correto da pilha para chamadas de sub-rotina, e construção de funções modulares. Também reforçou a importância de seguir convenções como uso de a0 para retorno e ra para controle de fluxo, principalmente quando se trabalha com chamadas encadeadas.

Apesar das dificuldades iniciais com o gerenciamento da pilha, a experiência foi bastante produtiva e contribuiu muito para a compreensão da estrutura e funcionamento de programas em linguagem de máquina.